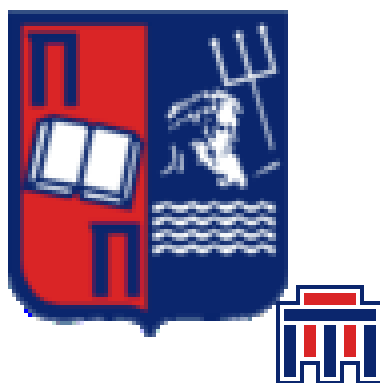


ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ

Τμήμα Πληροφορικής



Εργασία Μαθήματος *Ασφάλεια Λογισμικού*

Τίτλος Εργασίας	Εργασία 3: Ανάπτυξη Ηλεκτρονικού Καταστήματος (mini-eshop)
Όνομα φοιτητή – Αρ. Μητρώου	Γεώργιος Γκεζέρης – ΜΠΚΕΔ2509
Ημερομηνία παράδοσης	Ιούνιος 2026



Περιεχόμενα

Εισαγωγή	2
1 Λειτουργικότητα κάθε αρχείου πηγαίου κώδικα	3
1.1 Backend – auth/ (αυθεντικοποίηση, login, rate limiting)	3
1.2 Backend – catalog/ (κατάλογος προϊόντων & αναζήτηση)	3
1.3 Backend – cart/ (καλάθι στον server)	3
1.4 Backend – order/ (ολοκλήρωση παραγγελίας, review, email διαχειριστή)	4
1.5 Backend – config/ (ασφάλεια, session, web, logging)	4
1.6 Backend – shared/ & σημείο εισόδου	5
1.7 Frontend (frontend/src/)	5
1.8 Υποδομή & βάση δεδομένων	5
2 Διαχείριση δεδομένων συνόδου (session)	6
3 Αποτροπή επιθέσεων user enumeration στη σελίδα Login	7
4 Προστασία των κωδικών χρηστών στη βάση δεδομένων	8
5 Παράμετροι SQL και προστασία από SQL injection	9
6 Προστασία από CSRF	10
7 Προστασία από XSS (ποια λειτουργικότητα και ποια μέτρα)	11
8 Σημαντικά ρίσκα χωρίς υλοποιημένο μέτρο προστασίας	12
Παράρτημα – εκτέλεση της εφαρμογής	13

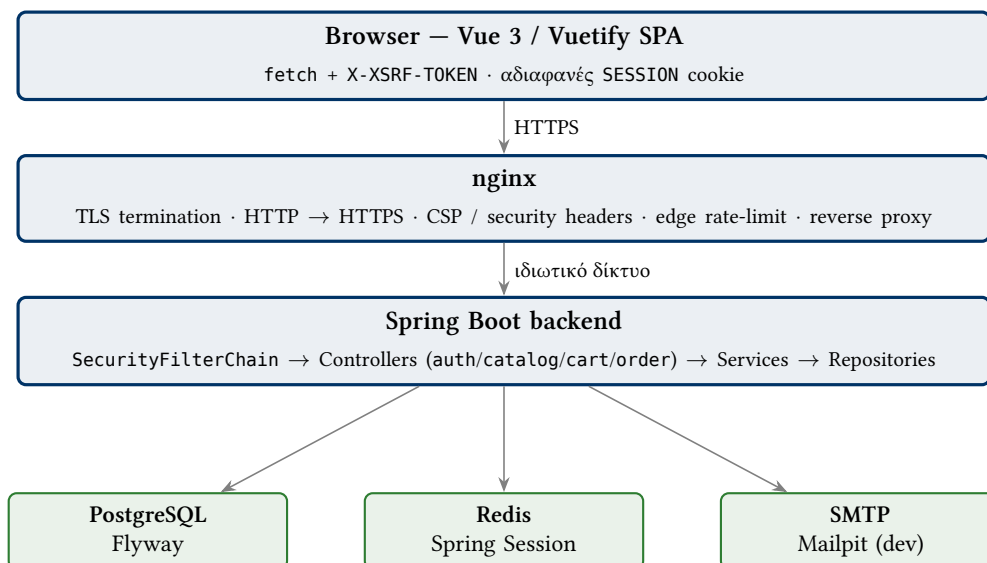
Εισαγωγή

Το **mini-eshop** είναι ένα ασφαλές ηλεκτρονικό κατάστημα, σχεδιασμένο με άμυνα σε βάθος (defense in depth) σε κάθε στρώμα. Tech stack:

- **Spring Boot 4** — REST backend.
- **Vue 3 / Vuetify** — SPA frontend.
- **nginx** — τερματισμός TLS, reverse proxy, εξυπηρέτηση στατικών αρχείων.
- **PostgreSQL** — μόνιμη αποθήκευση δεδομένων (Flyway migrations).
- **Redis** — αποθήκευση session (Spring Session).

Η εφαρμογή εκτελείται με την εντολή `make up`, που χτίζει τα images και ανεβάζει όλα τα containers μέσω Docker Compose· τα compose files βρίσκονται στο `deployments/`, με αναλυτικές οδηγίες στο `README.md`.

Η Εικόνα 1 δίνει τη συνολική εικόνα. Κάθε αίτημα ταξιδεύει από το **browser** (Vue SPA) μέσω του **nginx** — που τερματίζει το TLS, προσθέτει τα security headers/CSP, εφαρμόζει edge rate-limiting και κάνει reverse proxy — προς το **Spring Boot backend**. Εκεί περνά πρώτα από το `SecurityFilterChain` (εξουσιοδότηση, CSRF, session) και κατόπιν στους controllers ανά τομέα (`auth`, `catalog`, `cart`, `order`), οι οποίοι καλούν τα αντίστοιχα services και repositories. Η μόνιμη κατάσταση αποθηκεύεται στην **PostgreSQL** (σχήμα και δεδομένα μέσω Flyway), τα δεδομένα του session στο **Redis** (Spring Session), και οι επιβεβαιωμένες παραγγελίες ειδοποιούν τον διαχειριστή μέσω SMTP.



Εικόνα 1. Συνολική αρχιτεκτονική του mini-eshop και η ροή ενός αιτήματος, από το browser ως τα data stores.

Το παρόν έγγραφο απαντά στα οκτώ σημεία τεκμηρίωσης που ζητά η εκφώνηση. Κάθε αναφορά τεκμηριώνεται με παραπομπή στην αντίστοιχη θέση του πηγαίου κώδικα (`path:line`).



1 Λειτουργικότητα κάθε αρχείου πηγαίου κώδικα

Οι πίνακες που ακολουθούν περιγράφουν τη λειτουργικότητα κάθε αρχείου, ομαδοποιημένοι κατά πακέτο.

1.1 Backend – auth/ (αυθεντικοποίηση, login, rate limiting)

Αρχείο	Λειτουργικότητα
AuthController.java	REST endpoints για login: POST /api/auth/login (rate-limit → αυθεντικοποίηση → ανανέωση του session ID → αποθήκευση του security context) και GET /api/auth/me (επιστρέφει τον τρέχοντα χρήστη· η ανάλυση του CsrfToken εξαναγκάζει την εγγραφή του cookie XSRF-TOKEN).
AppUserDetailsService.java	UserDetailsService — φορτώνει χρήστη με το δοθέν username από τη βάση· ρίχνει UsernameNotFoundException αν δεν υπάρχει.
AppUser.java	JPA entity για τον πίνακα users (username, password_hash, όνομα, email).
AppUserRepository.java	Spring Data repository: findByUsername.
LoginRateLimiter.java	Throttling login βασισμένο σε Redis: μετρητές αποτυχιών ανά username και ανά IP με κυλιόμενο παράθυρο.
LoginRateLimitProperties.java	Typed config (app.login-rate-limit): κατώφλια + παράθυρο.
LoginRequest.java / UserInfo.java	DTOs αιτήματος / απάντησης login.
CurrentUser.java	Annotation @CurrentUser → εισάγει τον αυθεντικοποιημένο principal στα ορίσματα των controllers.

1.2 Backend – catalog/ (κατάλογος προϊόντων & αναζήτηση)

Αρχείο	Λειτουργικότητα
CatalogController.java	GET /api/products — σελιδοποιημένη λίστα + προαιρετικός όρος search (@Size(max=100)).
CatalogService.java	Επιλέγει μεταξύ «όλα τα προϊόντα» και «αναζήτηση κατά όνομα»· αντιστοιχίζει σε ProductSummary.
ProductRepository.java	Spring Data queries: findByNameContainingIgnoreCase, findByReference.
Product.java	JPA entity για products (περιλαμβάνει UUID reference, τιμή, νόμισμα).
ProductSummary.java / CatalogResponse.java	DTOs μόνο για ανάγνωση (δεν εκθέτουν ποτέ το αριθμητικό id της βάσης· χρησιμοποιούν το reference).

1.3 Backend – cart/ (καλάθι στον server)

Αρχείο	Λειτουργικότητα
CartController.java	POST /api/cart/items, προβολή/ενημέρωση/αφαίρεση — το καλάθι ζει στον server, συνδεδεμένο με τον χρήστη.
CartService.java	Επιχειρησιακοί κανόνες καλαθιού: εύρεση προϊόντος μέσω reference, προσθήκη/ενημέρωση ποσότητας, επιβολή policy.



CartItem.java / CartItemEntity.java	JPA entities (carts, cart_items). Τα items αποθηκεύουν μόνο productReference + quantity — ποτέ τιμή.
CartRepository.java	findByUser_Id.
CartPolicy.java	Όρια (μέγιστα διακριτά είδη / μέγιστη ποσότητα).
AddItemRequest / UpdateItemRequest / CartItem / CartResponse	DTOs καλαθιού.
CartFullException / CartItemQuantityException	Εξαιρέσεις domain → αντιστοιχίζονται σε 4xx.

1.4 Backend — order/ (ολοκλήρωση παραγγελίας, review, email διαχειριστή)

Αρχείο	Λειτουργικότητα
OrderController.java	POST /api/order/submit (δημιουργία review) και POST /api/order/confirm (οριστικοποίηση παραγγελίας).
OrderService.java	Ξαναδιαβάζει τις τιμές από τη βάση κατά το submit (αποτροπή price-tampering), αποθηκεύει παραγγελία PENDING, στο confirm την σημειώνει confirmed και δημοσιεύει event.
OrderEmailSender.java	Μετά το commit, στέλνει την παραγγελία με email στον διαχειριστή (απλό κείμενο, SimpleMailMessage).
ShippingAddress.java	DTO διεύθυνσης με validation, περιλαμβάνει προστασία CRLF — άμυνα έναντι email-header-injection.
OrderReview.java / OrderLine.java	Payload review + τιμή ανά γραμμή (η τιμή λαμβάνεται από το entity Product, όχι από τον client).
OrderEntity.java / OrderLineEntity.java	JPA entities (orders, order_lines).
OrderRepository.java	findByUser_IdAndStatus.
OrderConfirmedEvent.java	Event που δημοσιεύεται μετά το commit της συναλλαγής checkout.
OrderProperties.java	Typed config: διευθύνσεις email διαχειριστή/αποστολέα (validated @Email).

1.5 Backend — config/ (ασφάλεια, session, web, logging)

Αρχείο	Λειτουργικότητα
SecurityConfig.java	Η αλυσίδα φίλτρων ασφαλείας: εξουσιοδότηση (default-deny), CSRF (.spa()), session concurrency, logout, password encoder, AuthenticationManager, στρατηγική session-authentication.
SessionConfig.java	Serializer session για Redis — JSON (όχι JDK), περιορισμένος από PolymorphicTypeValidator.
SessionProperties.java	Typed config: absolute timeout του session.
AbsoluteSessionTimeout Interceptor.java	Ακυρώνει session παλαιότερο του απόλυτου ορίου, ανεξαρτήτως δραστηριότητας.
JsonAuthHandlers.java	AuthenticationEntryPoint + AccessDeniedHandler → JSON 401/403 (χωρίς redirects για το SPA).
ProblemDetailWriter.java	Γράφει απαντήσεις JSON ProblemDetail (RFC-9457).
GlobalExceptionHandler.java	Αντιστοιχίζει εξαιρέσεις σε απαντήσεις· συμπύσσει όλες τις AuthenticationException σε γενικό 401.



WebConfig.java	Καταχωρεί τον interceptor absolute-timeout και τον resolver ορίσματος @CurrentUser.
SecurityEventLogger.java	Δομημένη καταγραφή των events επιτυχίας/αποτυχίας αυθεντικοποίησης του Spring Security.

1.6 Backend – shared/ & σημείο εισόδου

Αρχείο	Λειτουργικότητα
EshopApplication.java	Σημείο εισόδου Spring Boot.
shared/LogFields.java	Σταθερές για κλειδιά/events δομημένης καταγραφής.
shared/PageMeta.java	DTO μεταδεδομένων σελιδοποίησης.

1.7 Frontend (frontend/src/)

Αρχείο	Λειτουργικότητα
App.vue	Κέλυφος + εναλλαγή προβολών SPA (Login / Catalog / Cart / Order)· badge καλαθιού στο app-bar· χειρισμός auth:expired.
views/LoginView.vue	Φόρμα login → POST /auth/login.
views/CatalogView.vue	Πλέγμα προϊόντων, αναζήτηση, ένδειξη «όλα vs αποτελέσματα αναζήτησης», add-to-cart, και <v-footer> με σύνοψη πλήθους καλαθιού.
views/CartView.vue	Προβολή περιεχομένων καλαθιού.
views/OrderView.vue	Μηχανή βημάτων checkout: φόρμα διεύθυνσης → review → ολοκλήρωση (χωρίς αλλαγή σελίδας).
api/client.js	Wrapper fetch: εισάγει το header CSRF στα mutating αιτήματα, χειρίζεται 401 → auth:expired.
main.js / plugins/vuetify.js	Bootstrap εφαρμογής και ρύθμιση Vuetify.

1.8 Υποδομή & βάση δεδομένων

Αρχείο	Λειτουργικότητα
nginx/nginx.conf	Τερματισμός TLS, ανακατεύθυνση HTTP→HTTPS, security headers/CSP, edge rate limiting, reverse proxy στο backend, SPA fallback.
nginx/gen-cert.sh	Παράγει το self-signed πιστοποιητικό.
nginx/Dockerfile	Build εικόνας nginx.
deployments/ (docker-compose*.yaml) + Makefile	Ενορχήστρωση υπηρεσιών (nginx, backend, postgres, redis, mail): base stack συν dev/prod overlays, με targets make up/dev/prod.
db/migration/V1__schema.sql	Πίνακες: users, products, orders, order_lines.
db/migration/V2__seed.sql	Seed δεδομένων προϊόντων (μέσω Flyway).
db/migration/V3__cart.sql	Πίνακες carts / cart_items.
scripts/seed-users.sql	Χειροκίνητο seed των demo χρηστών (alice/bob), εφαρμόζεται μέσω make seed· δεν εκτελείται από το Flyway.



2 Διαχείριση δεδομένων συνόδου (session)

Η αυθεντικοποίηση βασίζεται σε **stateful, server-side session** με υπόβαθρο **Spring Session + Redis**: το state του session (το `SecurityContext`) αποθηκεύεται στον server, στο Redis, ενώ ο browser κρατά μόνο ένα αδιαφανές session cookie με όνομα **SESSION** που το αναφέρει. Δεν χρησιμοποιείται JWT (stateless token).

Παραγωγή & ανανέωση (rotation) του session ID. Το login δεν χρησιμοποιεί ενσωματωμένο filter· το `formLogin` και το `httpBasic` είναι απενεργοποιημένα (`SecurityConfig.java:103-104`). Έτσι, το `AuthController.login` καλεί ρητά τη `SessionAuthenticationStrategy` (`AuthController.java:52`), η οποία είναι μια `CompositeSessionAuthenticationStrategy` (`SecurityConfig.java:164-173`) που εκτελεί, με τη σειρά:

1. `ConcurrentSessionControlAuthenticationStrategy` — επιβάλλει το όριο του ενός session·
2. `ChangeSessionIdAuthenticationStrategy` — **ανανεώνει (rotation) το session ID κατά το login (άμυνα έναντι session-fixation)**·
3. `RegisterSessionAuthenticationStrategy` — καταχωρεί το νέο session.

Το ίδιο το session ID παράγεται από τον servlet container / Spring Session (κρυπτογραφικά τυχαίο), όχι από κώδικα της εφαρμογής.

Χαρακτηριστικά cookie (`application.yml:76-78`): `http-only: true, secure: true, same-site: strict`. Έτσι το cookie δεν διαβάζεται από JavaScript, στέλνεται μόνο μέσω HTTPS, και δεν επισυνάπτεται σε cross-site αιτήματα.

Λήξεις — δύο ανεξάρτητα όρια:

- **Idle timeout 15m** — `server.servlet.session.timeout: 15m` (`application.yml:79`)· επανεκκινεί με τη δραστηριότητα.
- **Absolute timeout 8h** — `app.session.absolute-timeout: 8h` (`application.yml:6`), επιβάλλεται από το `AbsoluteSessionTimeoutInterceptor.java:37-42`, που ακυρώνει session παλαιότερο του ορίου **ανεξαρτήτως δραστηριότητας** (περιορίζει την αξία ενός κλεμμένου cookie).

Ταυτόχρονα sessions. Το `maximumSessions(1)` (`SecurityConfig.java:75-87`) με ένα `SpringSessionBackedSessionRegistry` (`SecurityConfig.java:153-157`, απαιτεί το indexed Redis repository, `application.yml:22-28`). Η δημιουργία ενός δεύτερου session **οδηγεί στον τερματισμό του παλαιότερου**· το επόμενο αίτημα εκείνου λαμβάνει 401 (`SecurityConfig.java:86`).

Logout (`SecurityConfig.java:90-102`): το `POST /api/auth/logout` ακυρώνει το session και εκπέμπει `Clear-Site-Data: cookies`.

Serialization. Τα δεδομένα του session αποθηκεύονται στο Redis ως JSON, όχι **JDK-serialized** (`SessionConfig.java:20-23`), και η αποσειριοποίηση περιορίζεται από έναν `BasicPolymorphicTypeValidator` σε συνδυασμό με τα Jackson modules του Spring Security (`SessionConfig.java:25-32`) — κλείνοντας την επιφάνεια επίθεσης `deserialization-gadget`.



3 Αποτροπή επιθέσεων user enumeration στη σελίδα Login

Η απάντηση του login είναι **πανομοιότυπη** είτε **υπάρχει το username** είτε **όχι**, σε τρεις άξονες:

1. **Ίδιο σφάλμα, ίδιο status.** Το AppUserDetailsService ρίχνει UsernameNotFoundException για άγνωστο χρήστη (AppUserDetailsService.java:22-25). Ο τυπικός DaoAuthenticationProvider (SecurityConfig.java:128) διατηρεί το hideUserNotFoundExceptions στην **προεπιλογή true**, οπότε ο άγνωστος χρήστης αναφέρεται ως BadCredentialsException — η ίδια εξαίρεση με τον λανθασμένο κωδικό. Στη συνέχεια το GlobalExceptionHandler.java:20-28 αντιστοιχίζει **κάθε** AuthenticationException σε γενικό 401 Unauthorized ProblemDetail χωρίς διακριτική λεπτομέρεια. (Πρόκειται για την προεπιλογή του framework, όχι για ρητή ρύθμιση μέσω setter.)
2. **Σχεδόν σταθερός χρόνος απόκρισης.** Ο ίδιος τυπικός provider εκτελεί **εικονικό υπολογισμό hash κωδικού όταν ο χρήστης δεν βρεθεί** (mitigateAgainstTimingAttack του Spring), ώστε «ανύπαρκτος χρήστης» και «λάθος κωδικός» να διαρκούν συγκρίσιμο χρόνο — κανένα timing oracle.
3. **Η απόκριση 429 δεν αποκαλύπτει την ύπαρξη λογαριασμού.** Ο LoginRateLimiter ελέγχεται **πριν** την αυθεντικοποίηση (AuthController.java:38) και, σε αποτυχία, αυξάνει **και τους δύο** μετρητές (ανά username και ανά IP) **συμμετρικά** — **ανεξαρτήτως αν υπάρχει το username** (LoginRateLimiter.java:43-45, καλείται από AuthController.java:62). Επομένως ο επιτιθέμενος δεν μπορεί να διακρίνει πραγματικά από ψεύτικα usernames μέσω της συμπεριφοράς throttling. Κατώφλια: 10/username, 20/IP, παράθυρο 15 λεπτών (LoginRateLimitProperties.java:13-15).



4 Προστασία των κωδικών χρηστών στη βάση δεδομένων

Οι κωδικοί αποθηκεύονται μόνο ως hashes Argon2id, ποτέ σε καθαρό κείμενο ή αναστρέψιμη μορφή.

- **Encoder** (SecurityConfig.java:142-150): ένας DelegatingPasswordEncoder με προεπιλεγμένο id argon2@SpringSecurity_v5_8, με υπόβαθρο Argon2PasswordEncoder.defaultsForSpringSecurity_v5_8(). Πρόκειται για τις **προεπιλεγμένες παραμέτρους Argon2id του Spring Security v5.8** (δεν ορίστηκαν προσαρμοσμένες memory/iterations/parallelism). Το πρόθεμα {argon2@SpringSecurity_v5_8} σε κάθε αποθηκευμένο hash καθιστά εφικτή μελλοντική αναβάθμιση αλγορίθμου χωρίς να θιγούν οι υπάρχουσες εγγραφές.
- **Αποθήκευση** (db/migration/V1__schema.sql:4): password_hash VARCHAR(255) NOT NULL. Ο αλγόριθμος Argon2 ενσωματώνει τυχαίο salt σε κάθε παραγόμενο hash (μέσα στην κωδικοποιημένη συμβολοσειρά), οπότε δεν χρειάζεται ξεχωριστή στήλη salt.
- **Επαλήθευση**: το login περνά από τον DaoAuthenticationProvider (SecurityConfig.java:128-129), που καλεί PasswordEncoder.matches(raw, storedHash) — ο ακατέργαστος κωδικός γίνεται hash και συγκρίνεται· το αποθηκευμένο hash δεν αποκρυπτογραφείται ποτέ.



5 Παράμετροι SQL και προστασία από SQL injection

Κάθε ερώτημα στη βάση είναι ένα **Spring Data derived query** (ερώτημα βάσει ονόματος μεθόδου). Δεν χρησιμοποιούνται χειροκίνητα γραμμένα SQL ερωτήματα, ούτε **@Query**, ούτε **native queries** σε κανένα repository — οπότε το Spring Data / Hibernate παράγει **παραμετροποιημένα prepared statements** και δεσμεύει κάθε τιμή· τα δεδομένα που προέρχονται από τον χρήστη δεν ενσωματώνονται ποτέ απευθείας στο κείμενο του SQL ερωτήματος.

Repository (file:line)	Μέθοδος / παράμετρος	Τρόπος δέσμευσης / ασφάλεια
AppUserRepository. java:8	findByUsername(String) — login username	δεσμευμένη παράμετρος σε derived query
ProductRepository. java:12	findByNameContaining IgnoreCase — όρος search (@Size(max=100))	δεσμευμένη παράμετρος LIKE· το %...% το προσθέτει το Hibernate, όχι string concat
ProductRepository. java:14	findByReference(UUID) — product reference	τύπου UUID· μη-UUID είσοδος απορρίπτεται πριν εκτελεστεί οποιοδήποτε ερώτημα
CartRepository.java:9	findByUser_Id(Long) — userId από τον principal (όχι είσοδος αιτήματος)	δεσμευμένη παράμετρος
OrderRepository.java:9	findByUser_ IdAndStatus(Long, String) — userId από principal· το status είναι η σταθερά εφαρμογής "PENDING" (OrderService.java:26,59,76), ποτέ είσοδος αιτήματος	δεσμευμένες παράμετροι

Έτσι, οι μόνες τιμές **από αίτημα χρήστη** που φτάνουν στην SQL είναι το **username του login**, ο **όρος αναζήτησης**, και το **product reference (UUID)** — όλες δεσμευμένες ως παράμετροι prepared-statement.



6 Προστασία από CSRF

Η προστασία CSRF είναι **ενεργοποιημένη** μέσω του preset SPA του Spring Security: `.csrf(CsrfConfigurer::spa)` (`SecurityConfig.java:89`). Αυτό ρυθμίζει:

- ένα `CookieCsrfTokenRepository` — το token παραδίδεται στον browser σε αναγνώσιμο cookie `XSRF-TOKEN`, και
- έναν `SpaCsrfTokenRequestHandler` — ο server περιμένει το token πίσω στο header `X-XSRF-TOKEN` (το κλασικό μοτίβο `double-submit-cookie` για SPAs).

Πλευρά frontend (`api/client.js:1-25`): για κάθε αίτημα εκτός GET/HEAD, ο client διαβάζει το cookie `XSRF-TOKEN` και στέλνει την τιμή του ως header `X-XSRF-TOKEN`. αν το token λείπει, αρνείται να στείλει το αίτημα. Το `GET /api/auth/me` αναλύει όρισμα `CsrfToken` (`AuthController.java:68-69`) ειδικά για να εξαναγκάσει την (επαν)έκδοση του cookie.

Γιατί είναι ασφαλές παρότι το cookie XSRF είναι αναγνώσιμο από JavaScript: ένας cross-site επιτιθέμενος **δεν μπορεί να διαβάσει** το cookie `XSRF-TOKEN` του θύματος (Same-Origin Policy) και άρα δεν μπορεί να θέσει το αντίστοιχο header `X-XSRF-TOKEN`. Η άμυνα ενισχύεται από το ότι το session cookie είναι `SameSite=strict` (`application.yml:78`), που ήδη εμποδίζει τον browser να επισυνάπτει credentials σε cross-site αιτήματα. Τα mutating endpoints είναι μόνο POST, οπότε απλό CSRF μέσω GET δεν μπορεί να τα ενεργοποιήσει.



7 Προστασία από XSS (ποια λειτουργικότητα και ποια μέτρα)

Σε κάθε σημείο όπου προβάλλονται δεδομένα προερχόμενα από χρήστες ή από τη βάση δεδομένων — τα αποτελέσματα καταλόγου/αναζήτησης, το καλάθι, και η οθόνη review παραγγελίας (CatalogView.vue, CartView.vue, OrderView.vue) — η έξοδος προστατεύεται από:

1. **Αυτόματο escaping του framework.** Όλο το δυναμικό περιεχόμενο χρησιμοποιεί text interpolation του Vue (`{{ }}` / `v-text`), που κάνει HTML-escape από προεπιλογή. Δεν υπάρχει `v-html` πουθενά στο frontend (επαληθεύτηκε: `grep -rn "v-html" frontend/src` → δεν βρέθηκε καμία αντιστοιχία), οπότε δεν υπάρχει sink ακατέργαστης HTML.
2. **Content-Security-Policy + headers θωράκισης** (nginx/nginx.conf:47-53), σε κάθε απάντηση: `default-src 'self', script-src 'self'` (καμία inline/απομακρυσμένη script μπλοκάρει injected `<script>`), `object-src 'none', base-uri 'self', frame-ancestors 'none', upgrade-insecure-requests`, καθώς και `X-Content-Type-Options: nosniff` και `X-Frame-Options: DENY`. (Το `style-src` επιτρέπει `'unsafe-inline'` μόνο επειδή το Vueify εισάγει styles κατά την εκτέλεση — δεν επιτρέπει inline scripts.)
3. **Έξοδος προς το email διαχειριστή — άμυνα header-injection.** Το email παραγγελίας είναι το μόνο σημείο όπου δεδομένα χρήστη φεύγουν από το web context. Τα πεδία διεύθυνσης αποστολής επικυρώνονται με `@Pattern(regex = "^[^\\r\\n]*$")` και στα τέσσερα πεδία (order/ShippingAddress.java:8-11), αποτρέποντας CR/LF email-header injection, και το θέμα του email καθαρίζεται επιπλέον από CR/LF (OrderEmailSender.java:75,85-87). Το σώμα του email είναι απλό κείμενο, όχι HTML, οπότε δεν αποτελεί sink HTML-XSS.



8 Σημαντικά ρίσκα χωρίς υλοποιημένο μέτρο προστασίας

Πρόκειται για γνωστά και συνειδητά αποδεκτά υπολειπόμενα ρίσκα ασφαλείας, τα οποία καταγράφονται ρητά αντί να αγνοούνται:

1. **Η μεταφορά του email παραγγελίας δεν επιβάλλει TLS στο προεπιλεγμένο (dev) προφίλ.** Το email προς τον διαχειριστή είναι απλό κείμενο SimpleMailMessage (OrderEmailSender.java:52-78). Στο προεπιλεγμένο προφίλ ο προορισμός είναι το τοπικό dev sink **Mailpit**, που δεν διαθέτει TLS, οπότε η μεταφορά δεν κρυπτογραφείται — εκ σχεδιασμού για την επίδειξη. Το προφίλ prod (application-prod.yml, ενεργό μέσω make prod) **επιβάλλει** mail.smtp.auth=true και mail.smtp.starttls.required=true, ώστε η εφαρμογή να *αρνείται* να στείλει αν ο SMTP server δεν διαπραγματεύεται TLS. Το υπολειπόμενο ρίσκο περιορίζεται έτσι στο dev προφίλ.
2. **Χωρίς κλείδωμα λογαριασμού / CAPTCHA.** Η άμυνα brute-force είναι μόνο rate-limiting (LoginRateLimiter) συν τα edge limits του nginx. Δεν υπάρχει διαρκές κλείδωμα λογαριασμού ή πρόκληση επαλήθευσης ανθρώπου, οπότε μια αργή κατανεμημένη επίθεση κάτω από τα κατώφλια δεν σταματά.
3. **Edge case INCR/EXPIRE του rate-limiter.** Μια κατάρρευση ανάμεσα στο Redis INCR και το EXPIRE θα μπορούσε να αφήσει μετρητή χωρίς TTL (ήδη σημειωμένο στο LoginRateLimiter.java). Αποδεκτό για το single-instance εύρος· χωρίς θωράκιση.
4. **Self-signed πιστοποιητικό.** Σύμφωνα με την εκφώνηση, το TLS χρησιμοποιεί αυτο-παραγόμενο πιστοποιητικό (nginx/gen-cert.sh), που δεν είναι CA-trusted — οι browsers προειδοποιούν, και δεν προστατεύει από ενεργό MITM που παρουσιάζει δικό του μη-έμπιστο cert το οποίο ο χρήστης αποδέχεται.
5. **Παραδοχές single-instance.** Ορισμένοι μηχανισμοί υποθέτουν μία instance backend· η οριζόντια κλιμάκωση θα χρειαζόταν επανεξέταση (το Redis ήδη βοηθά, αλλά δεν έγινε load-test).
6. **Δεν χρησιμοποιείται WAF ούτε μηχανισμός προστασίας από αυτοματοποιημένες επιθέσεις** σε ορισμένες μη-mutating επιφάνειες — εκτός εύρους για αυτή την εργασία, αλλά αξίζει να αναφερθεί.



Παράρτημα — εκτέλεση της εφαρμογής

Η εφαρμογή τρέχει σε containers μέσω Docker Compose, με βοηθητικό Makefile. Συνοπτικά βήματα (πλήρεις οδηγίες στο README.md):

1. `cp .env.example .env` και συμπλήρωση των `DB_PASSWORD` / `REDIS_PASSWORD`.
2. `make certs` — παραγωγή του self-signed πιστοποιητικού.
3. `make up` — build και εκκίνηση του stack: nginx (443, self-signed), backend, PostgreSQL (Flyway migrations + seed *προϊόντων*), Redis, και τοπικό mail sink Mailpit.
4. `make seed` — φόρτωση των demo χρηστών (alice/alicepass, bob/bobpass). *δεν* δημιουργούνται από το Flyway.

Πλοηγείστε στο <https://mini-eshop.local/> (αποδεχτείτε το self-signed cert) και συνδεθείτε με έναν από τους demo λογαριασμούς.

Το `make` δεν είναι απαραίτητο — κάθε target αντιστοιχεί σε μία εντολή `docker compose` (ο πίνακας αντιστοιχίας βρίσκεται στο README.md).